

1.Enable CloudTrail monitoring and store the events in S3 and CloudWatch log events.

Objective: Configure AWS CloudTrail to:

- Capture all AWS API activities.
- Store CloudTrail logs in an Amazon S3 bucket.
- Send CloudTrail events to Amazon CloudWatch Logs.

Prerequisites

- AWS Account
- IAM User with AdministratorAccess
- AWS Region: us-east-1 (or your preferred region)

Step 1: Create an S3 Bucket

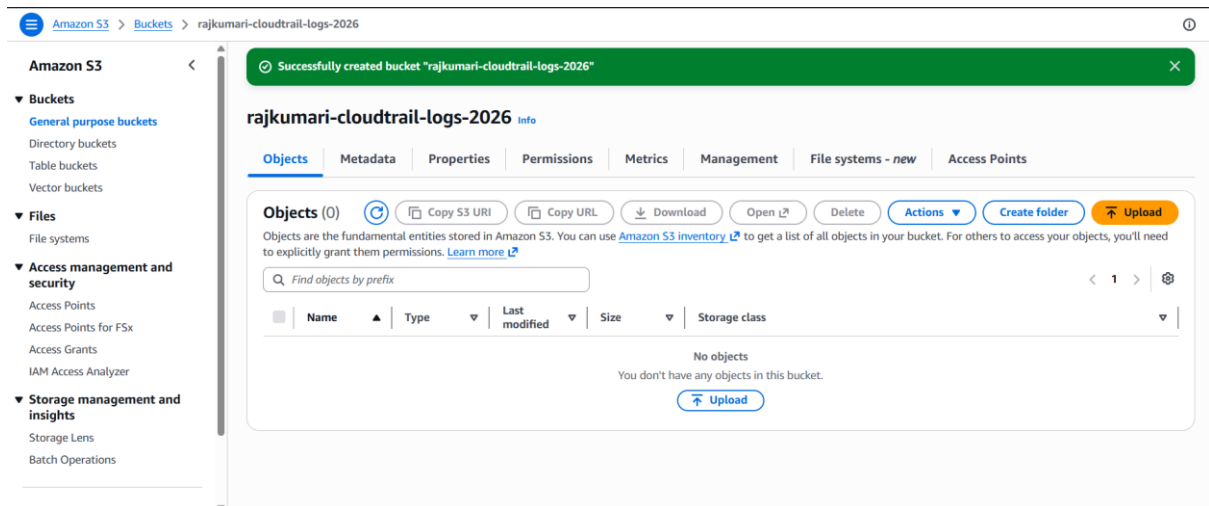
1. Login to AWS Console.
2. Search **S3**.
3. Click **Create bucket**.

Enter:

Setting	Value
Bucket Name	rajkumari-cloudtrail-logs-2026 (must be globally unique)
AWS Region	us-east-1
Block Public Access	Keep Enabled
Bucket Versioning	Optional

Setting	Value
Default Encryption	SSE-S3

Click **Create Bucket**.



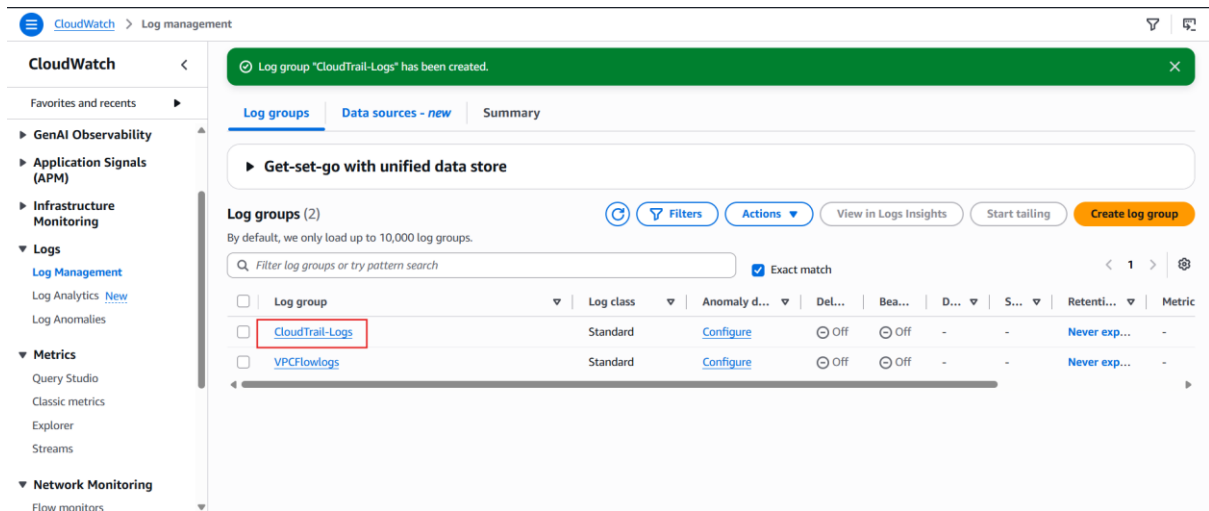
Step 2: Create a CloudWatch Log Group

1. Open **CloudWatch**.
2. Click **Logs**.
3. Select **Log Groups**.
4. Click **Create Log Group**.

Name: CloudTrail-Logs

Retention: Never Expire

Click **Create**.



Step 3: Create IAM Role for CloudTrail

Open: IAM

Click: Roles

Click: Create Role

Trusted Entity

Choose: AWS Service

Select: CloudTrail

Click: Next

Attach Permission

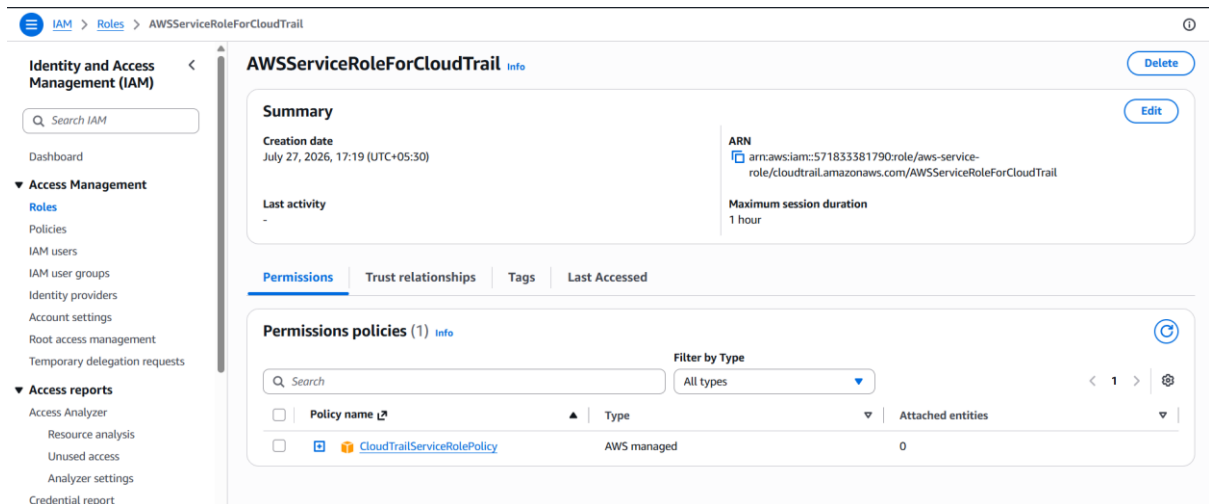
Search: CloudTrailServiceRolePolicy

Select it.

Click Next

Role Name: AWSServiceRoleForCloudTrail

Click: Create Role



Step 4: Open CloudTrail

Search

CloudTrail

Click

Trails

Click

Create Trail

Step 5: Configure CloudTrail

Trail Name

MyCloudTrail

Storage Location

Create new S3 bucket

or

Choose existing bucket

Select

rajkumari-cloudtrail-logs-2026

Leave

Log File SSE Encryption

Enabled.

Step 6: Configure Event Logging

Management Events

✓ Read

✓ Write

Data Events

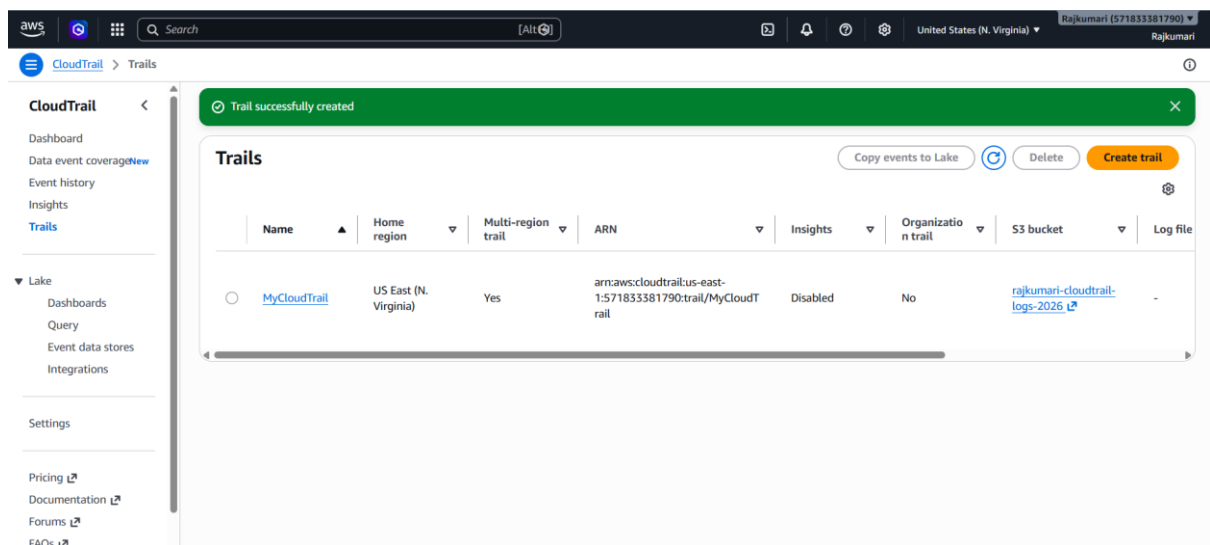
Skip

Insights

Skip

Click

Next



Step 7: Enable CloudWatch Logs

Enable

✓ CloudWatch Logs

Log Group

Choose

CloudTrail-Logs

IAM Role

Choose

CloudTrail-CloudWatch-Role

Click

Next

Step 8: Review

Review all settings.

Click

Create Trail

CloudTrail starts recording immediately.

Step 9: Verify Logs in S3

Go to

S3

Open

rajkumari-cloudtrail-logs-2026

Navigate to:

AWSLogs/

Account-ID/

CloudTrail/

Region/

Year/

Month/

Day/

You should see compressed log files:

The screenshot displays the Amazon S3 console interface. On the left, a navigation pane shows the 'Amazon S3' bucket structure, including 'Buckets', 'Files', 'Access management and security', and 'Storage management and insights'. The main content area shows the details for the object **571833381790_CloudTrail_us-east-1_20260727T1205Z_B76lsPebC0i1z3OM.json.gz**. The 'Properties' tab is active, showing the object's overview. The 'Object overview' section includes the Owner (8428a3e13f1864f9f3247847c2bc79dc9991e91c322156fbddfc4dbe1728e79), AWS Region (US East (N. Virginia) us-east-1), Last modified (July 27, 2026, 17:37:50 (UTC+05:30)), Size (929.0 B), Type (gz), and Key (AWSLogs/571833381790/CloudTrail/us-east-1/2026/07/27/571833381790_CloudTrail_us-east-1_20260727T1205Z_B76lsPebC0i1z3OM.json.gz). The right sidebar shows the S3 URI, Amazon Resource Name (ARN), Entity tag (Etag), and Object URL. Below the console, a browser window shows the raw log data in JSON format, which is a compressed CloudTrail log file.

```
{
  "Records": [
    {
      "eventVersion": "1.11",
      "userIdentity": {
        "type": "Root",
        "principalId": "571833381790",
        "arn": "arn:aws:iam::571833381790:root",
        "accountId": "571833381790",
        "accessKeyId": "ASIAV1K17M60PFY23AUA",
        "sessionContext": {}
      },
      "attributes": {
        "creationDate": "2026-07-27T05:38:53Z",
        "mfaAuthenticated": "true"
      },
      "eventTime": "2026-07-27T12:02:30Z",
      "eventSource": "s3.amazonaws.com",
      "eventName": "GetBucketPolicy",
      "awsRegion": "us-east-1",
      "sourceIPAddress": "189.168.27.187",
      "userAgent": [
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.84.0 Safari/537.36"
      ],
      "errorCode": "NoSuchBucketPolicy",
      "errorMessage": "The bucket policy does not exist.",
      "requestParameters": {
        "bucketName": "rajkumari-cloudtrail-logs-2026",
        "host": "rajkumari-cloudtrail-logs-2026.s3.us-east-1.amazonaws.com",
        "policy": ""
      },
      "responseElements": null,
      "additionalEventData": {}
    },
    {
      "signatureVersion": "SigV4",
      "cipherSuite": "TLS_CHACHA20_POLY1305_SHA256",
      "bytesTransferredIn": 0,
      "authenticationMethod": "AuthHeader",
      "x-amz-id-2": "d5a6d8302f64f9f3247847c2bc79dc9991e91c322156fbddfc4dbe1728e79",
      "bytesTransferredOut": 1343,
      "requestID": "F0E5MHWQX04EE81",
      "eventID": "66aa81e6-897d-4a71-84a1-c33b8e8ae7e",
      "readOnly": true,
      "resources": [
        {
          "accountId": "571833381790",
          "type": "AWS::S3::Bucket",
          "arn": "arn:aws:s3:::rajkumari-cloudtrail-logs-2026"
        }
      ],
      "eventType": "AwsApiCall",
      "managementEvent": true,
      "recipientAccountId": "571833381790",
      "eventCategory": "Management",
      "tlsDetails": {
        "tlsVersion": "TLSv1.3",
        "cipherSuite": "TLS_CHACHA20_POLY1305_SHA256",
        "clientProvidedHostHeader": "rajkumari-cloudtrail-logs-2026.s3.us-east-1.amazonaws.com"
      }
    }
  ]
}
```

Step 10: Verify Logs in CloudWatch

Open

CloudWatch

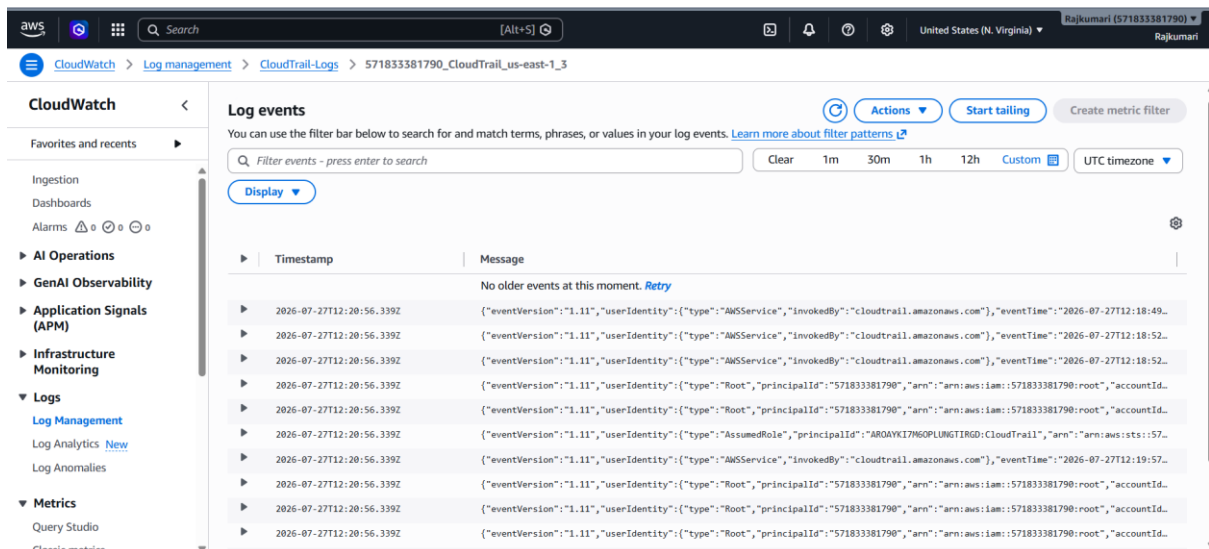
Go to

Logs

Open

CloudTrail-Logs

Open the latest **Log Stream**.



2. Enable SNS for CloudTrail to send alerts via email.

Objective: Configure **Amazon SNS (Simple Notification Service)** with **CloudTrail** so that whenever CloudTrail delivers a new log file, an email notification is sent to the subscribed email address.

Prerequisites

- AWS Account
- IAM User with AdministratorAccess
- Existing CloudTrail Trail
- Existing S3 Bucket for CloudTrail Logs
- Valid Email Address

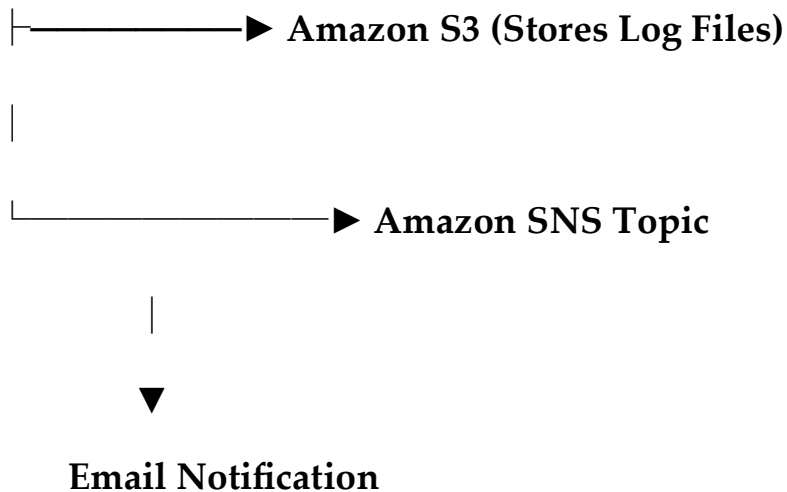
AWS API Activity

|



AWS CloudTrail

|



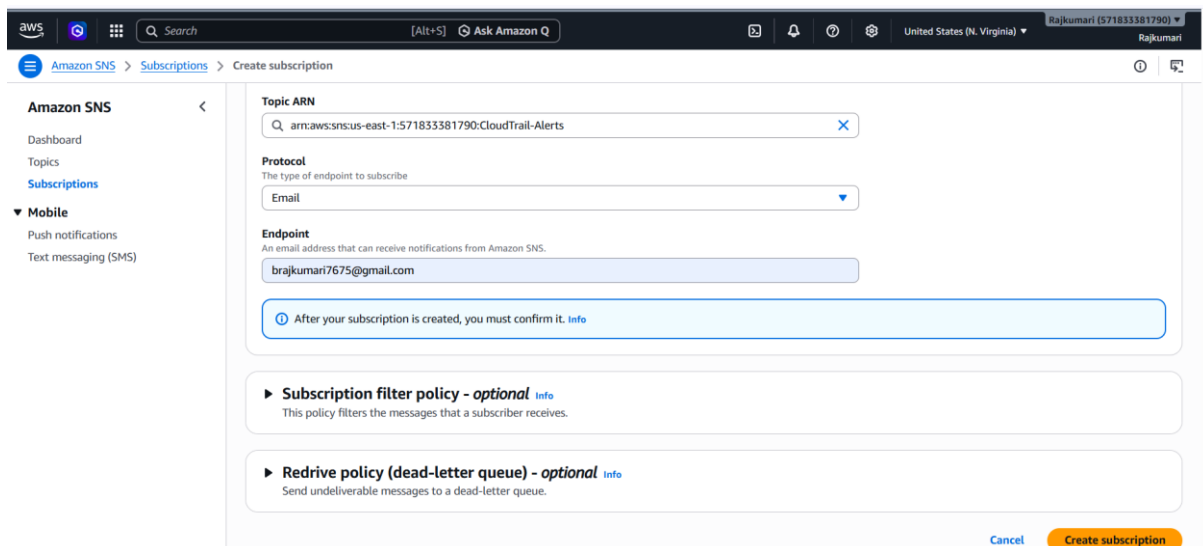
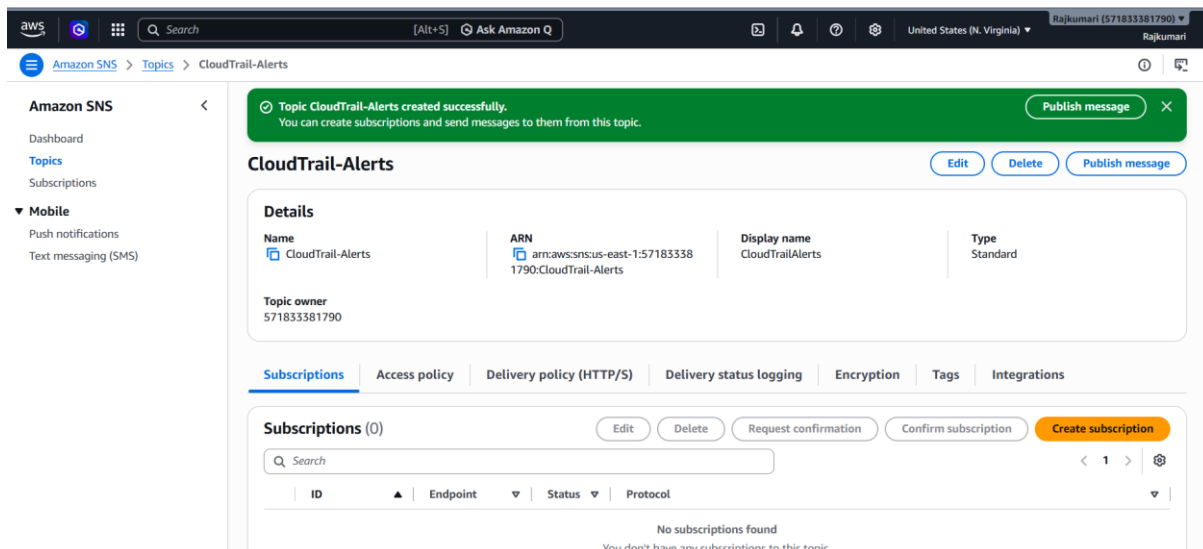
Step 1: Create an SNS Topic

1. Sign in to the **AWS Management Console**.
2. Search for **SNS**.
3. Click **Simple Notification Service**.
4. In the left pane, click **Topics**.
5. Click **Create topic**.

Configure the following:

Setting	Value
Type	Standard
Name	CloudTrail-Alerts
Display Name (Optional)	CloudTrailAlerts

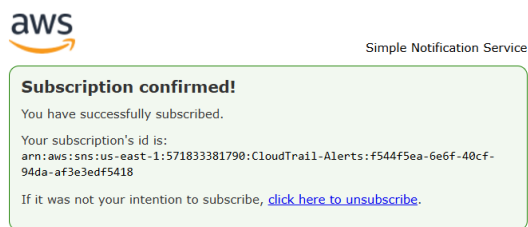
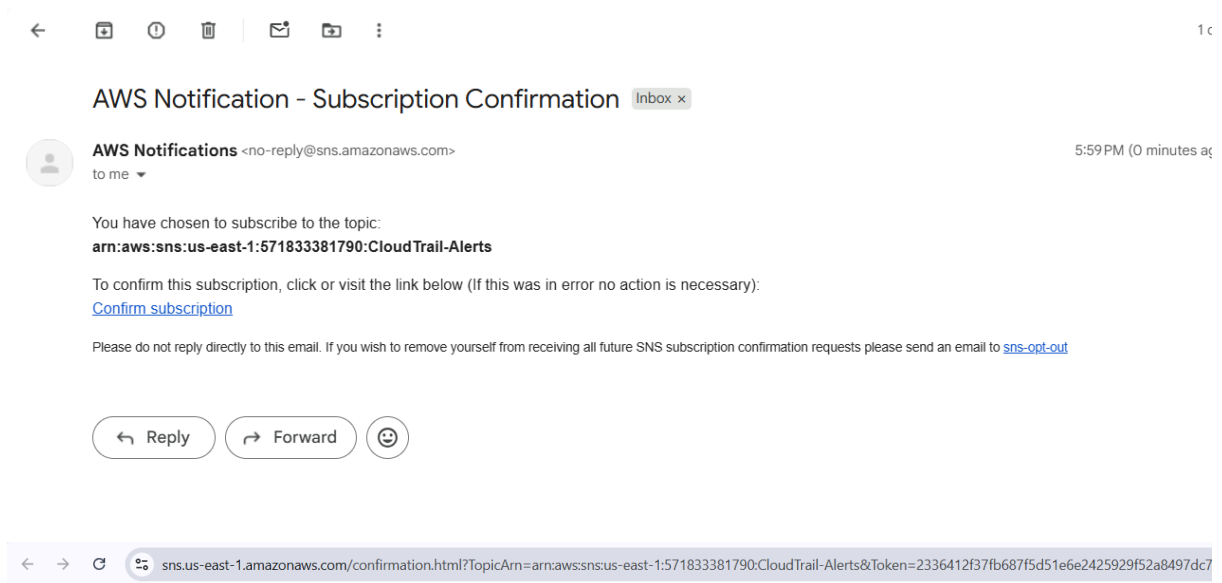
Click **Create topic**.



Step 3: Confirm the Subscription

1. Open your email inbox.
2. Look for an email from **AWS Notifications**.
3. Click **Confirm subscription**.

After confirmation, the subscription status changes from **Pending confirmation** to **Confirmed**.



Step 4: Configure CloudTrail

1. Open **CloudTrail**.
2. Click **Trails**.
3. Select your trail (**MyCloudTrail**).
4. Click **Edit**.

Step 5: Enable SNS Notifications

Scroll to the **SNS notification delivery** section.

Configure:

Setting	Value
SNS Notification Enable	

Setting	Value
SNS Topic	Existing
Topic Name	CloudTrail-Alerts

Click **Save changes**.

The screenshot shows the AWS CloudTrail console 'Edit' page for a trail named 'MyCloudTrail'. The left sidebar contains navigation links for CloudTrail, Trails, and various settings. The main content area is divided into sections for encryption, customer managed AWS KMS key, and additional settings. The 'Log file SSE-KMS encryption' is enabled. The 'Customer managed AWS KMS key' is set to 'Existing' with an AWS KMS alias of 'arn:aws:kms:us-east-1:571833381790:key/71fed58-08a1-4074-ad23-e6e5582d24d6'. Under 'Additional settings', 'Log file validation' and 'SNS notification delivery' are both enabled. The 'Create a new SNS topic' is set to 'Existing' with an SNS topic of 'arn:aws:sns:us-east-1:571833381790:CloudTrail-Alerts'. At the bottom right, there are 'Cancel' and 'Save changes' buttons.

Step 6: Generate CloudTrail Activity

Perform some AWS actions, such as:

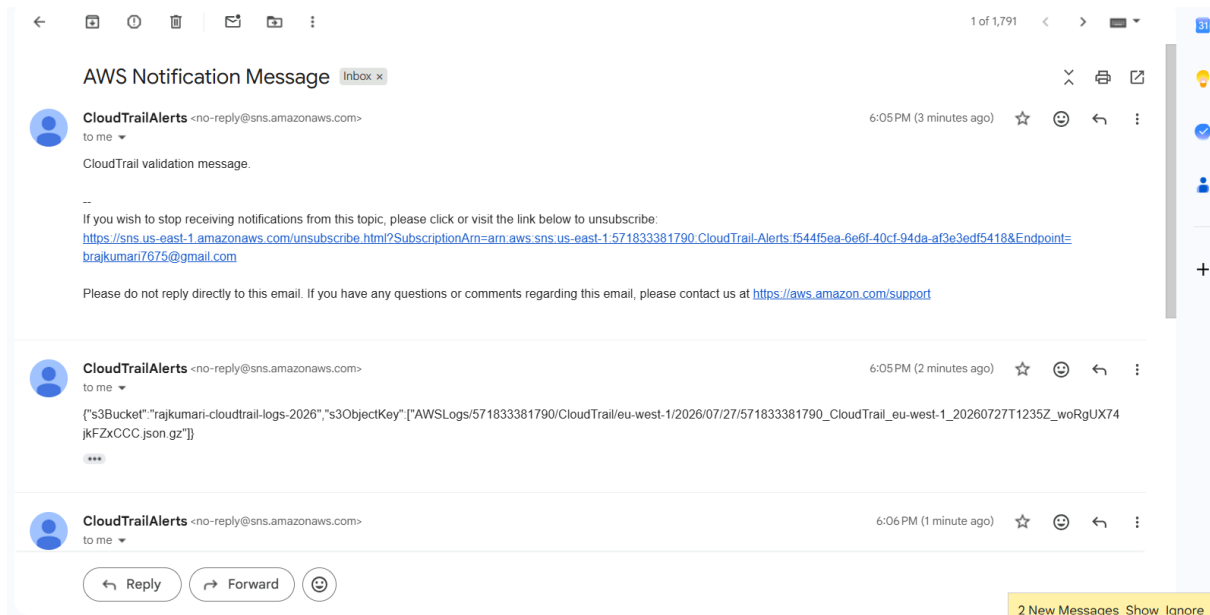
- Create an S3 bucket.
- Launch an EC2 instance.
- Create an IAM user.
- Stop or start an EC2 instance.

CloudTrail records these API calls.

Step 7: Verify Email Notification

Wait a few minutes.

Check your email inbox.



Step 8: Verify in the AWS Console

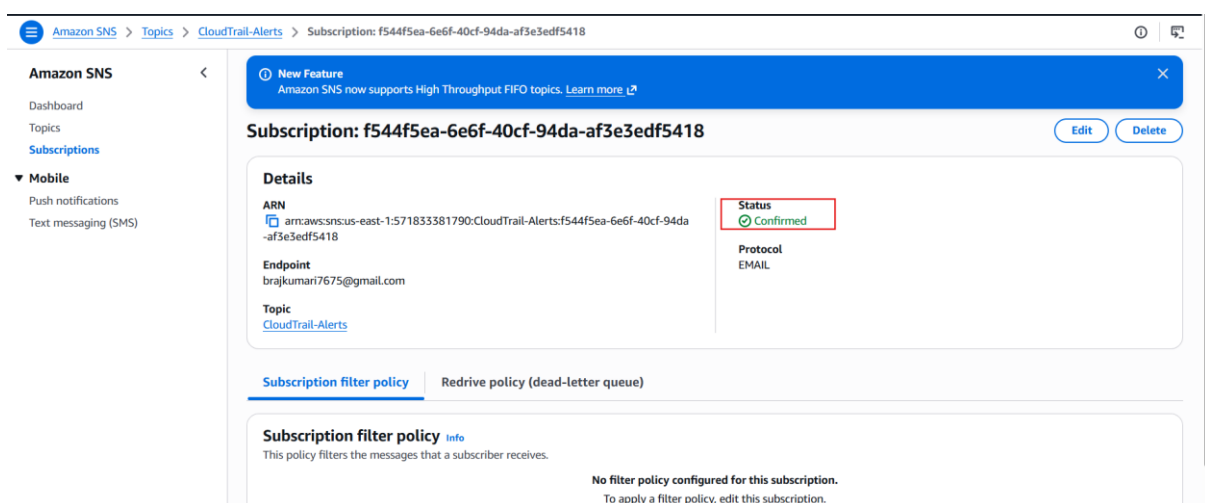
Verify SNS Subscription

Go to:

SNS → Topics → CloudTrail-Alerts → Subscriptions

Expected status:

Confirmed



Verify CloudTrail

Go to:

CloudTrail → MyCloudTrail

Expected:

SNS Notification Delivery

Enabled

AWS CLI Verification

```
[root@ip-172-31-18-188 ~]# aws cloudtrail describe-trails
{
  "trailList": [
    {
      "Name": "MyCloudTrail",
      "S3BucketName": "rajkumari-cloudtrail-logs-2026",
      "SnsTopicName": "arn:aws:sns:us-east-1:571833381790:CloudTrail-Alerts",
      "SnsTopicARN": "arn:aws:sns:us-east-1:571833381790:CloudTrail-Alerts",
      "IncludeGlobalServiceEvents": true,
      "IsMultiRegionTrail": true,
      "HomeRegion": "us-east-1",
      "TrailARN": "arn:aws:cloudtrail:us-east-1:571833381790:trail/MyCloudTrail",
      "LogFileValidationEnabled": true,
      "CloudWatchLogsLogGroupARN": "arn:aws:logs:us-east-1:571833381790:log-group:CloudTrail-Logs:*",
      "CloudWatchLogsRoleARN": "arn:aws:iam:571833381790:role/service-role/CloudTrail-CloudWatch-Role",
      "KmsKeyId": "arn:aws:kms:us-east-1:571833381790:key/71fef58-08a1-4074-ad23-e6e5582d24d6",
      "HasCustomEventSelectors": true,
      "HasInsightSelectors": false,
      "IsOrganizationTrail": false
    }
  ]
}
[root@ip-172-31-18-188 ~]# aws sns list-subscriptions
{
  "Subscriptions": [
    {
      "SubscriptionArn": "arn:aws:sns:us-east-1:571833381790:CloudTrail-Alerts:f544f5ea-6e6f-40cf-94da-af3e3edf5418",
      "Owner": "571833381790",
      "Protocol": "email",
      "Endpoint": "brajkumari7675@gmail.com",
      "TopicArn": "arn:aws:sns:us-east-1:571833381790:CloudTrail-Alerts"
    }
  ]
}
```

3. Configure CloudWatch monitoring and record the CPU utilization and other metrics of EC2.

Objective: Configure Amazon CloudWatch to monitor an EC2 instance and record performance metrics such as:

- CPU Utilization
- Network In
- Network Out
- Disk Read Bytes
- Disk Write Bytes
- Status Check Failed

Prerequisites

- AWS Account

- IAM User with AdministratorAccess
- Running EC2 Instance
- CloudWatch Agent (Optional, for memory and disk metrics)

Step 1: Launch an EC2 Instance

If you already have an EC2 instance, skip this step.

1. Open **EC2 Console**.
2. Launch an Amazon Linux 2023 instance.
3. Instance Type: t2.micro (or t3.micro)
4. Create/select a key pair.
5. Configure the Security Group.
6. Launch the instance.

Step 2: Verify Basic Monitoring

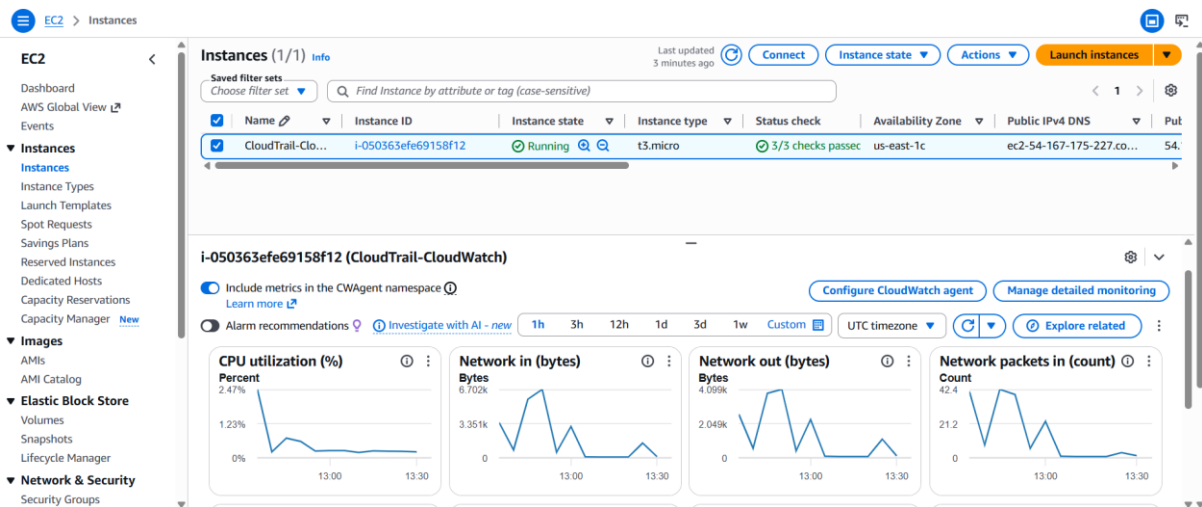
1. Open **EC2 Console**.
2. Select your EC2 instance.
3. Click the **Monitoring** tab.

You should see default CloudWatch metrics such as:

- CPU Utilization
- Network In
- Network Out
- Disk Read Bytes
- Disk Write Bytes

- Status Check Failed

Note: Basic monitoring collects metrics every **5 minutes**.



Step 3: Enable Detailed Monitoring (Optional)

1. Select the EC2 instance.
2. Click **Actions**.
3. Choose:

Monitor and troubleshoot

4. Click:

Manage detailed monitoring

5. Select:

Enable detailed monitoring

6. Save the changes.

Now CloudWatch collects metrics every **1 minute**.

Step 4: View Metrics in CloudWatch

1. Open **CloudWatch Console**.
2. Click **Metrics**.
3. Select:

All Metrics

4. Open:

EC2

5. Click:

Per-Instance Metrics

Select your instance.

Step 5: Monitor CPU Utilization

Locate the metric:

CPUUtilization

You can view:

- Average
- Minimum
- Maximum
- Sum

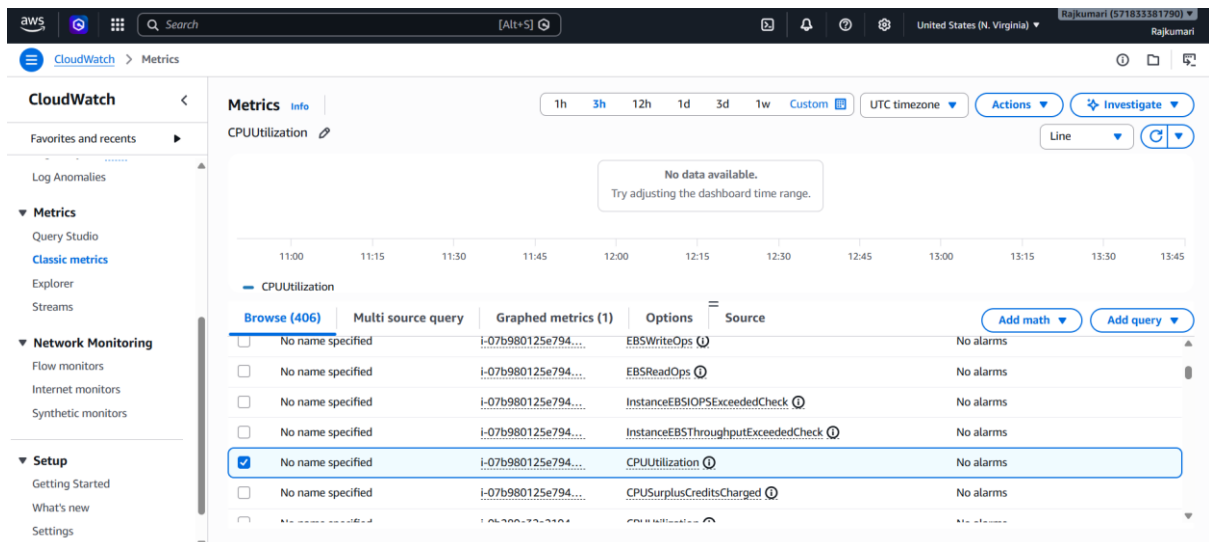
This graph shows the CPU usage percentage over time

Step 6: Monitor Other Metrics

Observe the following metrics:

Metric	Description
CPUUtilization	CPU usage percentage
NetworkIn	Incoming network traffic
NetworkOut	Outgoing network traffic
DiskReadBytes	Bytes read from disk

Metric	Description
DiskWriteBytes	Bytes written to disk
StatusCheckFailed	EC2 instance health status



Step 7: Generate CPU Load

Connect to your EC2 instance:

```
ssh -i key.pem ec2-user@<Public-IP>
```

Install the stress tool:

Amazon Linux 2023

```
sudo dnf install stress -y
```

Run a CPU stress test for 5 minutes:

```
stress --cpu 2 --timeout 300
```

This increases CPU utilization.

```

[root@ip-172-31-18-188 ~]# sudo dnf install stress -y
Amazon Linux 2023 Kernel Livepatch repository 519 kB/s | 61 kB 00:00
Dependencies resolved.
=====
Package      Architecture Version                      Repository      Size
=====
Installing:
stress       x86_64      1.0.7-2.amzn2023.0.1        amazonlinux     34 k
=====
Transaction Summary
=====
Install 1 Package

Total download size: 34 k
Installed size: 68 k
Downloading Packages:
stress-1.0.7-2.amzn2023.0.1.x86_64.rpm 871 kB/s | 34 kB 00:00
-----
Total                                     425 kB/s | 34 kB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
  Installing    : stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
  Running scriptlet: stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
  Verifying     : stress-1.0.7-2.amzn2023.0.1.x86_64 1/1

Installed:
stress-1.0.7-2.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-18-188 ~]# stress --cpu 2 --timeout 300
stress: info: [28557] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd

```

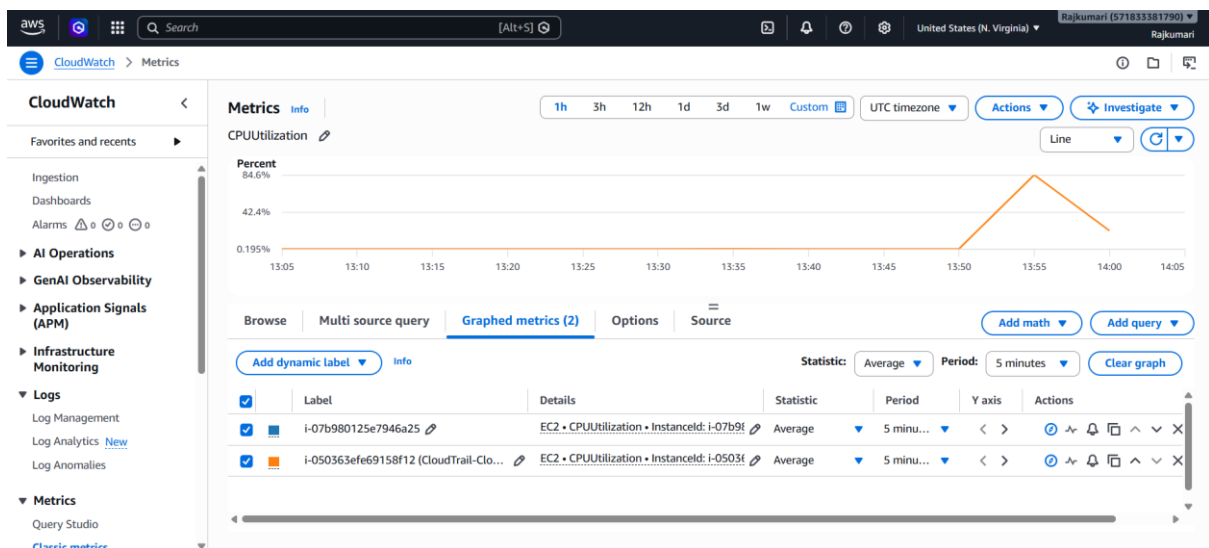
Step 8: Verify CPU Graph

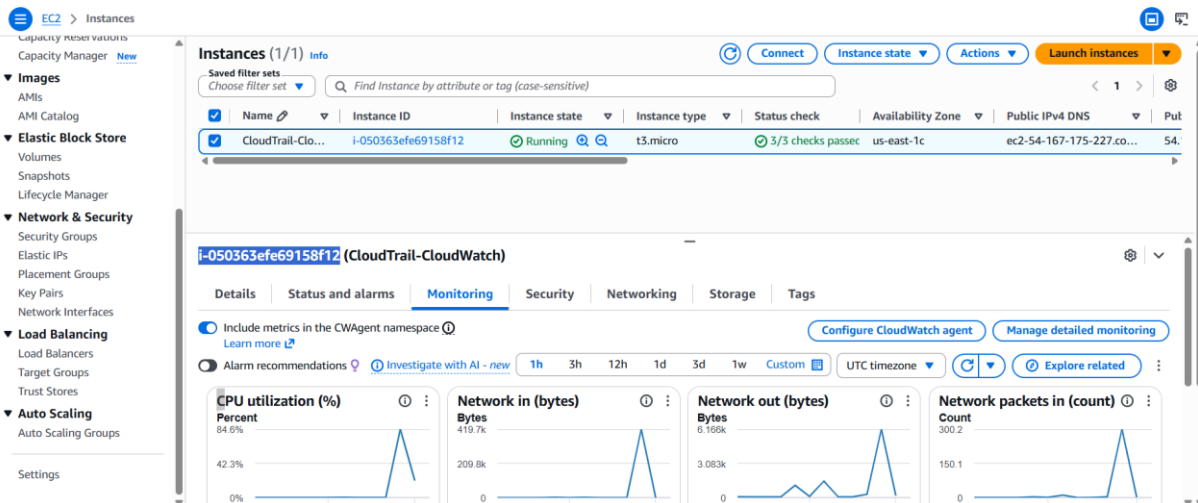
Go back to:

CloudWatch → Metrics → EC2 → Per-Instance Metrics

Refresh the graph after a few minutes.

You should observe an increase in **CPUUtilization**.





4. Create one alarm to send an alert to email if the CPU utilization is more than 70 percent.

Objective: Create a CloudWatch alarm that sends an email notification through Amazon SNS when the EC2 instance's CPU utilization is greater than 70%.

Step 1: Open CloudWatch

1. Sign in to the AWS Management Console.
2. Search for **CloudWatch**.
3. Open the **CloudWatch** service.

Step 2: Go to Alarms

In the left navigation pane:

CloudWatch → Alarms → All alarms

Click: **Create alarm**

Step 3: Select Metric

1. Click **Select metric**.
2. Choose:

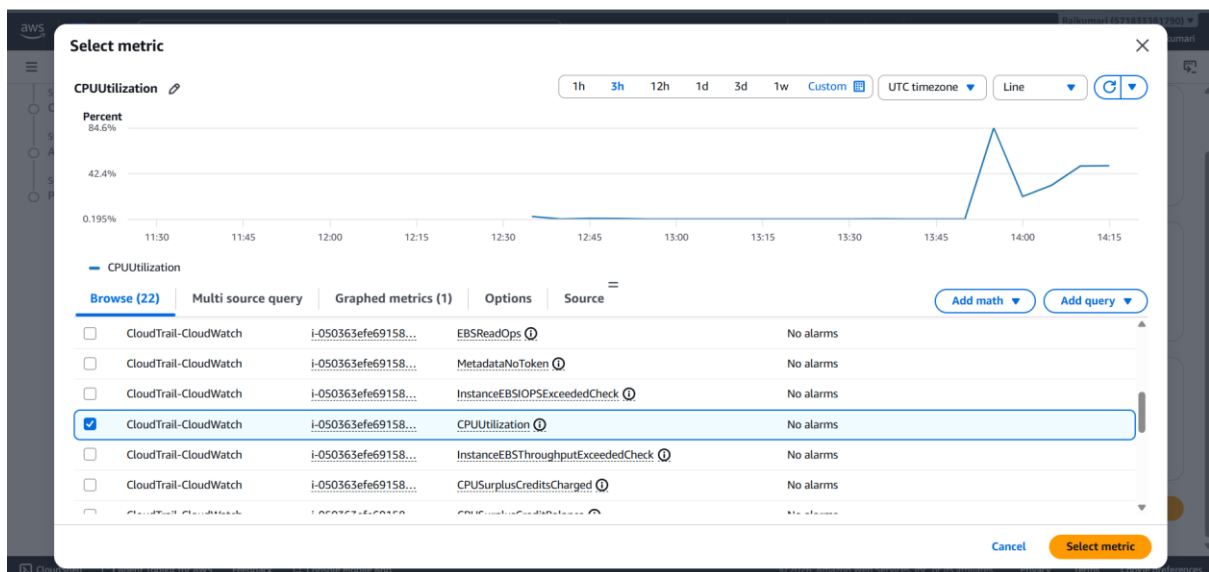
EC2

3. Click:

Per-Instance Metrics

4. Select your EC2 instance's **CPUUtilization** metric.

5. Click **Select metric**.



Step 4: Configure the Metric

Configure the alarm as follows:

Setting Value

Metric CPUUtilization

Statistic Average

Period 5 Minutes

Click **Next**.

Step 5: Configure the Threshold

Set the following values:

Setting **Value**

Threshold Type Static

Condition Greater

Threshold Value **70**

This means:

Trigger the alarm if the average CPU utilization is **greater than 70%**.

Click **Next**.

CloudWatch > Alarms > Create alarm

Step 1: Specify metric and conditions

Data source
Select the type of data to monitor

☒ Metrics
Monitor CloudWatch metrics

☐ Logs
Monitor CloudWatch log queries

☐ Alarms
Combine multiple alarms

Type
Choose the metric type for this alarm.

☒ Classic
Use the standard CloudWatch metric selection experience.

☐ PromQL
Use PromQL queries to define the metric for this alarm.

Metric

Graph
This alarm will trigger when the blue line goes above the red line for 1 datapoints within 5 minutes.

Percent: 84.6%

70

Namespace: AWS/EC2

Metric name:

Edit

CloudWatch > Alarms > Create alarm

Metric

Graph
This alarm will trigger when the blue line goes above the red line for 1 datapoints within 5 minutes.

Percent: 84.6%

70

42.4%

0.195%

11:30 12:00 12:30 13:00 13:30 14:00 14:30

CPUUtilization

Namespace: AWS/EC2

Metric name: CPUUtilization

InstanceId: i-050363efe69158f12

Instance name: CloudTrail-CloudWatch

Statistic: Average

Period: 5 minutes

Conditions

CloudWatch > Alarms > Create alarm

Conditions

Threshold type

☒ **Static**
Use a value as a threshold

☐ **Anomaly detection**
Use a band as a threshold

Whenever CPUUtilization is...
Define the alarm condition.

☒ **Greater**
> threshold

☐ **Greater/Equal**
>= threshold

☐ **Lower/Equal**
<= threshold

☐ **Lower**
< threshold

than...
Define the threshold value.

70

Must be a number.

► **Additional configuration**

Cancel Next

Step 6: Configure Notifications

Under **Notification**:

- **Alarm state trigger:** In alarm
- **Send a notification to the following SNS topic:**
 - Choose **Select an existing SNS topic**
 - Select your SNS topic (for example, CloudTrail-Alerts, or create a new topic such as EC2-CPU-Alerts if you prefer to keep it separate)

If you don't already have an email subscription:

1. Create a new SNS topic.
2. Add your email address.
3. Confirm the subscription from your email inbox.

Click **Next**.

CloudWatch > Alarms > Create alarm

Step 1: Specify metric and conditions
 Step 2: **Configure actions**
 Step 3: Add alarm details
 Step 4: Preview and create

Configure actions

Notification

Alarm state trigger
 Define the alarm state that will trigger this action.

☒ **In alarm**
 The metric or expression is outside of the defined threshold.

☐ **OK**
 The metric or expression is within the defined threshold.

☐ **Insufficient data**
 The alarm has just started or not enough data is available.

[Remove](#)

Send a notification to the following SNS topic
 Define the SNS (Simple Notification Service) topic that will receive the notification.

☒ **Select an existing SNS topic**
☐ Create new topic
☐ Use topic ARN to notify other accounts

Send a notification to...

CloudTrail-Alerts

Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.

Email (endpoints)
 brajkumari7675@gmail.com - [View in SNS Console](#)

[Add notification](#)

Step 8: Review and Create

Review all the settings.

Click:

Create alarm

CloudWatch > Alarms

Successfully created alarm EC2-High-CPU-Alarm. [View alarm](#)

Alarms | Mute rules - new

Alarms (1)

Filter alarms

Name	State	Actions	Actions muted - new	Last state update (UTC)	Condition
EC2-High-CPU-Alarm	Insufficient data	Actions enabled	-	2026-07-27 14:39:02	CPUUtiliza minutes

Step 9: Verify the Alarm

Go to:

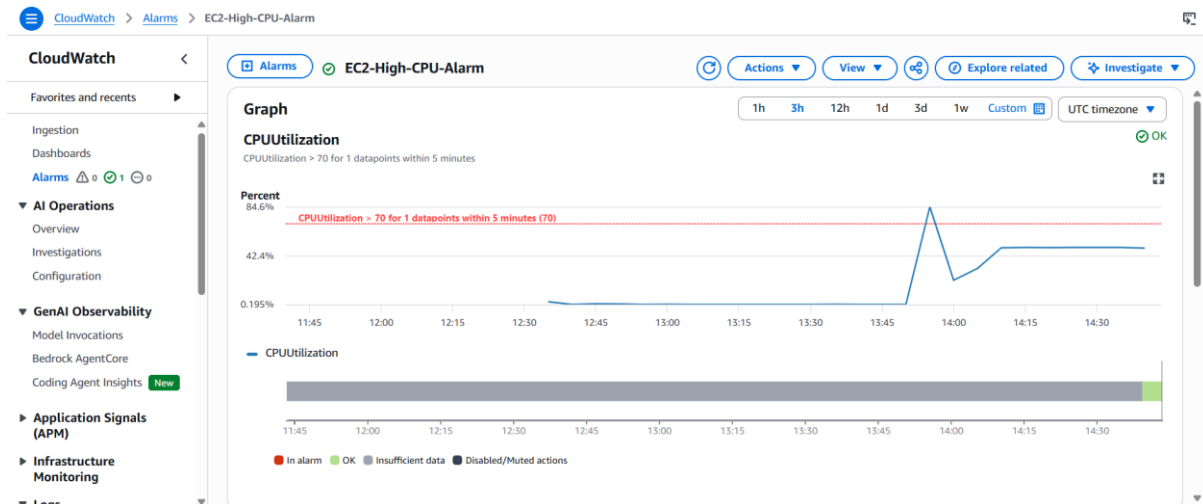
CloudWatch → Alarms

Initially, the alarm state will be: ok

After CPU utilization stays above **70%** for the configured evaluation period, the state changes to: **ALARM**

You should also receive an email notification from Amazon SNS.

When the CPU utilization drops below the threshold, the alarm automatically returns to the **OK** state.



5. Create a Dashboard and monitor the Tomcat service whether it is running or not and send the alert

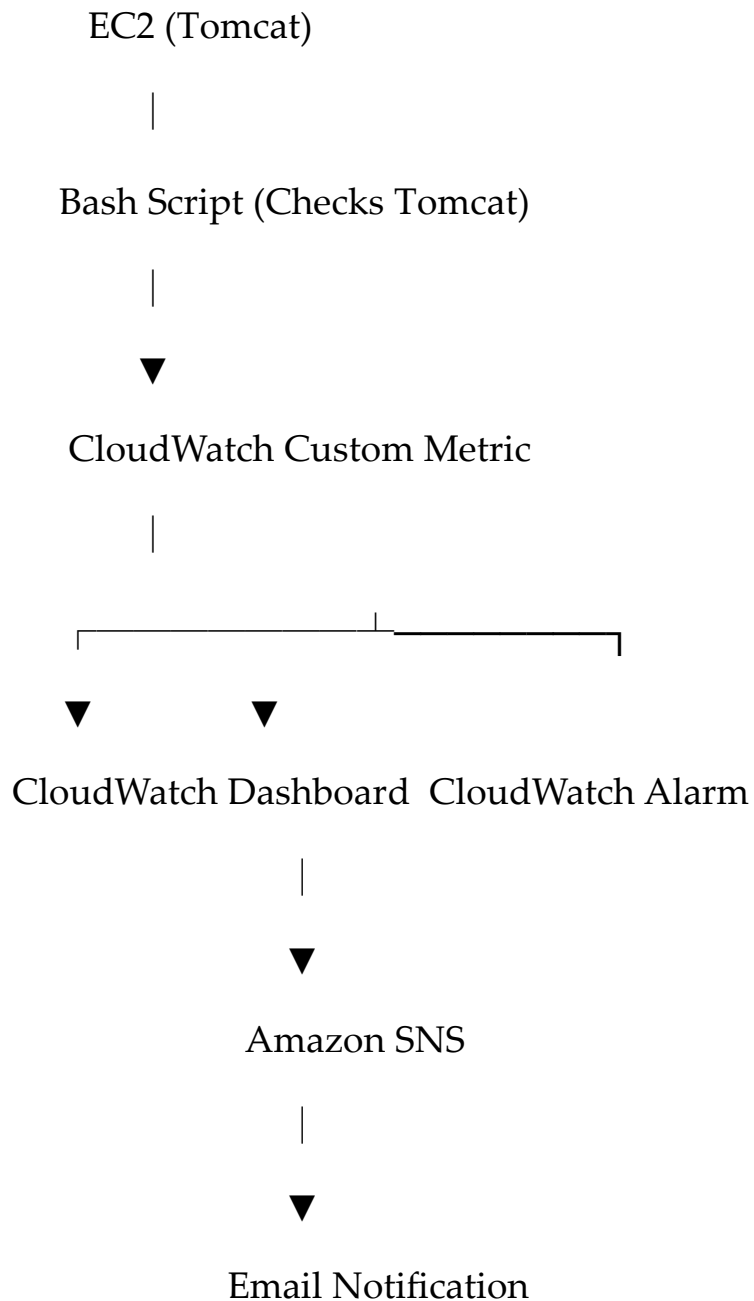
Objective

- Create a CloudWatch Dashboard.
- Monitor whether the Tomcat service is **running**.
- Send an email alert if the Tomcat service **stops**.

Prerequisites

- Running EC2 instance
- Apache Tomcat installed
- Amazon CloudWatch Agent installed
- IAM Role attached to EC2 with:
 - CloudWatchAgentServerPolicy
 - AmazonSSMManagedInstanceCore (recommended)
- SNS Topic with confirmed email subscription

Architecture



Step 1: Verify Tomcat

Connect to the EC2 instance: `$ sudo systemctl status tomcat`

If Tomcat is running, you should see: Active: active (running)

```
[root@ip-172-31-18-188 opt]# cd /opt/tomcat/bin
[root@ip-172-31-18-188 bin]# chmod +x *.sh
[root@ip-172-31-18-188 bin]# ./startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
[root@ip-172-31-18-188 bin]# |
```

Step 2: Create Tomcat Monitoring Script

Create a script that checks Tomcat status.

`sudo vi /home/ec2-user/tomcat_status.sh`

```
#!/bin/bash

STATUS=$(systemctl is-active tomcat)

if [ "$STATUS" = "active" ]
then
echo "1"
else
echo "0"
fi
~
```

Step 3: Install CloudWatch Agent

Install agent: `$ sudo yum install amazon-cloudwatch-agent -y`

Check: `sudo systemctl status amazon-cloudwatch-agent`

```
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-07-27 15:14:09 UTC; 29s ago
     Main PID: 31607 (amazon-cloudwat)
       Tasks: 7 (limit: 1059)
      Memory: 46.3M
         CPU: 602ms
    CGroup: /system.slice/amazon-cloudwatch-agent.service
            └─31607 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-clou

Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31613]: Executing /opt/aws/amazon-cloudwatch-agent/bin/a
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: D! [EC2] Found active network interface
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: I! imds retry client will retry 1 timesI! Detect
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: 2026/07/27 15:14:10 Reading json config file pat
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: /opt/aws/amazon-cloudwatch-agent/etc/amazon-clou
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: 2026/07/27 15:14:10 Reading json config file pat
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: 2026/07/27 15:14:10 I! Valid Json input schema.
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: I! Trying to detect region from ec2
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31618]: 2026/07/27 15:14:10 Configuration validation fir
Jul 27 15:14:10 ip-172-31-18-188.ec2.internal start-amazon-cloudwatch-agent[31607]: I! Detecting run_as_user...
```

6. Create a Dashboard and monitor the Nginx service to send the alert if Nginx is not running.

Step 1: Install Nginx

Connect to your EC2 instance:

Install Nginx: `$ sudo yum install nginx -y`

Step 2: Start Nginx Service

Start Nginx: `$ sudo systemctl start nginx`

Check status: `$sudo systemctl status nginx`

Step 3: Create Nginx Monitoring Script

Create a script: `$ sudo vi /home/ec2-user/nginx_status.sh`

Step 4: Configure CloudWatch Agent

Create CloudWatch Agent configuration:

```
root@ip-172-31-18-188:/opt/tomcat/bin

[root@ip-172-31-18-188 bin]# sudo vi /home/ec2-user/nginx_status.sh
[root@ip-172-31-18-188 bin]# chmod +x /home/ec2-user/nginx_status.sh
[root@ip-172-31-18-188 bin]# sudo vi /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
[root@ip-172-31-18-188 bin]# sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json \
-s
***** processing amazon-cloudwatch-agent *****
Starting config-downloader, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-downloader -download-source file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -output-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default -mode ec2]I! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
Start configuration validation...
2026/07/27 15:26:40 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_amazon-cloudwatch-agent.json.tmp ...
2026/07/27 15:26:40 I! Valid Json input schema.
2026/07/27 15:26:40 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default -input /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -input-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -output /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -mode ec2]I! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
Configuration validation succeeded
[root@ip-172-31-18-188 bin]# |
```

**CloudTrailAlerts** <no-reply@sns.amazonaws.com> 8:55 PM (2 minutes ago) ☆ 😊 ↶ ⋮
to me ▾
{"s3Bucket":"rajkumari-cloudtrail-logs-2026","s3ObjectKey":["AWSLogs/571833381790/CloudTrail/us-east-1/2026/07/27/571833381790_CloudTrail_us-east-1_20260727T1525Z_gpxvWmIpLMRYwI0D.json.gz"]}

**CloudTrailAlerts** <no-reply@sns.amazonaws.com> 8:57 PM (1 minute ago) ☆ 😊 ↶ ⋮
to me ▾
{"s3Bucket":"rajkumari-cloudtrail-logs-2026","s3ObjectKey":["AWSLogs/571833381790/CloudTrail/us-east-1/2026/07/27/571833381790_CloudTrail_us-east-1_20260727T1530Z_EMPDxTQqkNzc7H9p.json.gz"]}

**CloudTrailAlerts** <no-reply@sns.amazonaws.com> 8:57 PM (1 minute ago) ☆ 😊 ↶ ⋮
to me ▾
{"s3Bucket":"rajkumari-cloudtrail-logs-2026","s3ObjectKey":["AWSLogs/571833381790/CloudTrail/us-east-1/2026/07/27/571833381790_CloudTrail_us-east-1_20260727T1525Z_Q9clFonZiYuiKhYE.json.gz"]}

↶ Reply

↷ Forward

😊